

When Analyzers Know More Than They Tell: Evidence-Guided Refinement of LLM-Generated Vulnerability Checkers

ANONYMOUS AUTHOR(S)

Recent work shows that LLMs can synthesize static-analysis checkers from security patches, but a compilable checker is not necessarily a correct one. A patch reveals what changed, not why the change is safe, so generated checkers often match only the surface edit and miss the deeper semantic condition. Existing refinement loops feed back compilation errors and validation reports, yet these signals describe symptoms without explaining what the checker misunderstands about the code.

We observe that the analysis framework itself already computes the information needed to close this gap. Control-flow graphs, symbolic values, memory regions and checker callbacks are available inside the analyzer but unused by current refinement strategies. SEMWEAVER is a plug-in refinement layer that extracts patch-relevant analyzer evidence and assembles it with source context and validation feedback into a structured, provenance-labelled bundle. An iterative editing loop refines the checker on both the vulnerable and fixed revisions, guarded by a retention gate that accepts warning recovery or false-positive reduction while preventing over-pruning.

We evaluate SEMWEAVER on 39 KNighter-derived CSA checkers from Linux kernel fixes. With analyzer evidence, refinement reaches 23 warning-positive vulnerable revisions (vs. 15 for KNighter’s loop) and raises version-level F_1 from 0.448 to 0.575. Removing internal evidence yields nearly the same F_1 with fewer fixed-side reports. Additional experiments cover three editor models on a 12-subject subset, a CodeQL query refinement, and controlled source-form variations. A qualitative follow-up on all 11 non-tied comparisons shows how evidence helps repair checker recognition, and why the resulting warnings can still concern the wrong object, scope or path.

CCS Concepts: • **Software and its engineering** → **Automated static analysis**; • **Security and privacy** → **Systems security**.

Additional Key Words and Phrases: static analysis, vulnerability checkers, large language models, patch-guided refinement

ACM Reference Format:

Anonymous Author(s). 2027. When Analyzers Know More Than They Tell: Evidence-Guided Refinement of LLM-Generated Vulnerability Checkers. In *Proceedings of The ACM International Conference on the Foundations of Software Engineering (FSE 2027)*. ACM, New York, NY, USA, 22 pages.

1 Introduction

Static analysis is a cornerstone of software security assurance for systems code. Tools such as CodeQL [18] and Clang Static Analyzer (CSA) [37] detect vulnerabilities by checking source code against predefined rules without executing it [4, 7, 15, 49]. The effectiveness of these tools depends on the quality of their vulnerability checkers, each of which must encode the semantic conditions that distinguish a real bug from safe code. Writing such checkers by hand requires expertise in

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FSE 2027, Shenzhen, China

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

both the target codebase and the analysis framework, making manual authoring expensive and difficult to scale.

LLM-based patch-driven synthesis offers a promising alternative. In this paradigm, a security patch serves as a compact before-and-after specification: the LLM consumes the patch and synthesizes a checker that should fire on the vulnerable revision but remain silent on the fixed revision. KNighter [60], a representative system, has demonstrated that this approach can produce compilable and executable CSA checkers from historical Linux kernel fixes.

However, a compilable checker is not necessarily a correct one. The difficulty is that a patch shows what changed in the code, but not why the change makes the code safe. Consider a one-character fix that tightens a loop bound from $i < N$ to $i < N-1$: the real reason this works is that it prevents an array access at $i+1$ from exceeding the array’s capacity. A checker that merely looks for the old loop bound but does not understand the array access behind it will either report both versions or detect neither. Attempting to fix such a checker can also backfire: edits that suppress spurious reports may simultaneously remove the trigger for the true bug, a failure mode we call over-pruning.

A natural response is to iteratively refine the checker using feedback. Existing strategies, including KNighter’s own loop, feed compilation errors and validation reports back to the LLM so it can edit the checker [40, 53, 60]. Yet this feedback only describes symptoms. A message such as “the vulnerable side is silent” tells the model that something is wrong, but not what the checker misunderstands about the code. A cleanup attribute may name a lowered callback that the checker never recognizes. A cast may hide the expression expected by a callback. A state lookup may use a different region from the preceding update.

Our key observation is that **the analysis framework itself already computes the semantic information needed to close this gap, but existing refinement loops do not use it**. CSA, for example, exposes control-flow graphs, symbolic values, memory regions and checker callbacks. CodeQL provides similar depth through its program database and queries [2, 18]. This information can reveal why a checker misses a vulnerability, not just that it does. The challenge is selecting the relevant parts of this rich output and relating them to the patch and the checker under refinement.

We present SEMWEAVER, a refinement framework that bridges this gap by feeding analyzer-internal evidence into the LLM’s editing loop. Where existing refinement loops rely primarily on compilation and report feedback, SEMWEAVER additionally extracts semantic context from the analyzer itself: a rule-guided preflight selects patch-relevant views such as control-flow structure, symbolic-value bindings and checker callback traces, so the model can see what the checker actually observes at runtime rather than guess from silence alone. These facts are assembled with source context and validation feedback into a typed evidence bundle with explicit provenance, giving the model a structured diagnosis instead of a flat error log. The agent then edits the checker and validates it on both revisions, guarded by a retention gate that accepts warning recovery or false-positive reduction but rejects edits that lose an existing vulnerable-side warning, directly countering the over-pruning problem.

This paper makes the following contributions:

- We identify a gap between paired warning feedback and the analyzer’s representation of the code a generated checker inspects. We define analyzer-internal evidence by its backend origin, extraction interface and source binding, and study how it can guide edits to an existing checker rather than regenerate one.

- To address this gap, we build SEMWEAVER as an end-to-end plug-in refinement system. It selects patch-relevant views, joins analyzer, source and validation records in a typed, provenance-labelled bundle, and iteratively edits, compiles and scans the detector on both revisions. A retention gate accepts warning recovery or fixed-side report reduction without losing an existing vulnerable warning. The main implementation targets CSA, with a separate CodeQL query adapter.
- We then evaluate all 39 selected generated-only Linux CSA checkers from the same starting artifacts, comparing the actual KNighter refinement loop with native and no-internal refinement. Native recovers eight initially silent vulnerable revisions. At the same time, no-internal reaches nearly the same F_1 with fewer fixed-side reports overall. We also repeat the evidence comparison on 12 subjects, run three editor configurations three times on that subset, refine an archived CodeQL query in three decodes, and inspect all 11 non-tied comparisons alongside controlled source changes. These analyses show both useful recognition repairs and warnings that remain on the wrong object, scope or path.

2 Background

This section introduces the background on static vulnerability checkers, LLM-based checker generation from security patches, and the analyzer-internal intermediate results that motivate our approach.

2.1 Static Vulnerability Checkers

Static analysis examines source code without executing it, aiming to find bugs, security vulnerabilities, and code-quality issues at compile time or during code review [4, 15]. A vulnerability checker is a rule or query within a static-analysis framework that flags program locations matching a security-relevant condition. Modern frameworks support different checker paradigms. Path-sensitive engines such as CSA explore program paths, tracking branch guards, symbolic values and memory regions [37]. Relational query engines such as CodeQL express checkers as queries over extracted program facts [2, 18]. Both paradigms have been deployed at industrial scale [7, 49].

Even so, writing effective checkers remains a bottleneck. A checker must encode the conditions that distinguish a real bug from a safe use of the same operation. It must also express those conditions through the framework’s callbacks, values and object representations. The resulting warning burden affects practical use [20, 23]. Warning ranking and actionability prediction help users inspect reports [29, 48]; checker refinement changes the code that produces them.

2.2 LLM-Based Patch-Driven Checker Generation

A security patch provides a compact before-and-after specification: the vulnerable revision should trigger the generated checker, and the fixed revision should not. This idea builds on work that treats patches as vulnerability specifications, including semantic patching [43], clone-based vulnerability discovery [22, 28], and patch-derived datasets [16]. KNighter [60] synthesizes CSA checkers from historical Linux fixes and already refines them using compiler feedback, diagnostic and report-path context, and validation on both revisions. Its no-report branch provides no report-driven edit.

A patch describes a particular repair rather than every safe and unsafe use of an operation. The generated checker may therefore omit an important condition or encode it incorrectly. SEMWEAVER retains the patch-driven setting and adds analyzer context to the editing loop, including recovery attempts when the starting checker is silent.

2.3 Analyzer-Internal Intermediate Results

CSA maintains symbolic states and region bindings during path exploration; Clang also exposes control-flow and call-graph representations. CodeQL materializes program relations that can be queried for call and data-flow information [2, 18, 37]. Such representations have long supported taint analysis, graph-based detection and interprocedural reasoning [1, 25, 36, 58].

These facts can help explain how a generated checker interacts with its backend. For example, a callback observation can reveal the expression shape or region on which the checker actually operates. Their interpretation depends on their source: a compiler-produced record, an emitted diagnostic and a source window supply different kinds of information. Section 4.2 defines these origins and the interfaces used in our implementation.

3 Motivating Example

We illustrate the refinement process end to end on a single case: the KNightier-derived CSA checker for Linux commit 97cba232549b, which repairs an off-by-one access in the AMD display driver [52, 60]. This case is small enough to inspect manually yet exhibits the core problem: the generated checker recognizes a look-ahead access but misses the relation between the loop bound and the array's capacity.

The bug and its patch. The array `dc->links` has a declared capacity of `MAX_PIPES * 2`, abbreviated as N in Figure 1 (1). The function `get_host_router_total_dp_tunnel_bw` iterates over these links. On a matching host-router branch, the loop body reads both `links[i]` and `links[i + 1]` (M10–M11, 3). Under the original bound $i < N$ (M5, 2), the loop allows $i = N - 1$, so the look-ahead reaches index N and overflows the array. The patch tightens the bound to $i < N - 1$ (M6, 4), capping the largest look-ahead index at $N - 1$.

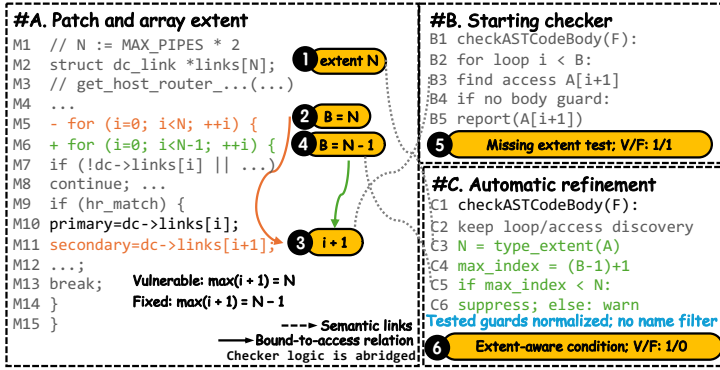


Fig. 1. **Motivating example.** An off-by-one look-ahead bug and its refinement. $N = \text{MAX_PIPES} * 2$. Alerts change from 1/1 to 1/0 after SEMWEAVER adds a capacity-aware condition.

Why the baseline checker fails. The generated checker already recognizes an induction variable, a loop bound, and a look-ahead subscript. It warns whenever the access lacks a protective condition in an enclosing `if` (B1–B5, 5). This rule checks only the branch guards and overlooks the protection carried by the loop bound itself. Because the fixed program still contains `links[i + 1]`, the checker fires on both revisions despite the tighter bound, yielding 1/1 alerts in independent replay. KNightier’s own refinement loop does not change this checker: its report-driven policy finds

no actionable feedback on this case, so the output remains 1/1. In short, the checker has learned that a look-ahead access needs a guard, but not that the loop condition already provides one.

Our refinement with analyzer evidence. SEMWEAVER starts from the same validation symptom (1/1) but goes further by assembling analyzer-internal context alongside the patch and source. Specifically, it collects four evidence records: two from Clang 18 internals (a CFG dump via `debug.DumpCFG` and a call graph via `debug.DumpCallGraph`) and two from analyzer output. The CFG exposes the loop structure and branch context that the checker misses. The patch and source records identify the bound change.

With this evidence, the LLM adds a capacity-aware condition to the existing `checkASTCodeBody` traversal. The refined checker reads the declared array extent from the AST type and computes the maximum look-ahead index: for a loop $i < B$, that index is $(B - 1) + 1 = B$. It suppresses the warning when B is smaller than the array extent N (C3–C6, ⑥). Under the vulnerable bound $B = N$, the maximum index equals the extent, so the warning fires. Under the fixed bound $B = N - 1$, the maximum index is safely below the extent, so the checker stays silent. Crucially, this condition is based on type and index relations rather than the patched function name, a specific capacity constant, or adjacency of statements. The first candidate passes independent Linux replay at 1/0.

Benefits over the baseline. The refined checker offers two concrete advantages over KNighter’s refined output. First, higher precision on the original Linux pair: SEMWEAVER’s checker correctly reports 1/0 (warns on the vulnerable revision, stays silent on the fixed one), whereas KNighter’s checker remains at 1/1 because it never learned the bound-to-capacity relation. Second, better generalization: on 114 controlled C programs built from the same look-ahead pattern (with varied names, capacities, unrelated statements, and loop-bound formulations), SEMWEAVER’s checker gives the expected result on all 114 executions (58 unsafe warned, 56 safe silent), while the checker produced by KNighter’s refinement loop correctly handles only 31 unsafe and 25 safe cases. AddressSanitizer independently confirms the safety labels for 80 of these variations [51]. Both advantages stem from the same root cause: analyzer evidence (CFG and call-graph context) lets the model understand why the loop bound matters for safety, so the refined checker encodes a semantic condition (maximum index vs. array extent) rather than matching syntactic patterns.

4 Approach

SEMWEAVER is a post-synthesis refinement layer for LLM-generated vulnerability detectors. Given a draft CSA checker or CodeQL query produced by an upstream generator, the patch that motivated the synthesis, and the corresponding source tree and validation scope, SEMWEAVER refines the detector to recover vulnerable-side warnings or reduce fixed-side reports.

Figure 2 summarizes the workflow across five stages. **(a) Generator context.** SEMWEAVER receives a draft detector, patch, source tree and validation target. It treats the generator as a black box and refines its output in place. **(b) Evidence assembly.** Rule-guided preflight selects evidence requirements, and four collectors gather patch facts, source context, analyzer evidence and validation feedback. **(c) Evidence bundle.** The facts are normalized into typed records with their origin and source bindings. **(d) Refinement.** The agent edits the latest attempt using a bounded view of those records. Paired validation updates a separate best retained detector. **(e) Audit artifacts.** Each run persists evidence, prompts, replies, candidates and validation results, so a refinement can be inspected or resumed. The audit links the retained checker to the evidence shown, attempted edits and paired outcomes; RQ3 compares refinement with and without native evidence.

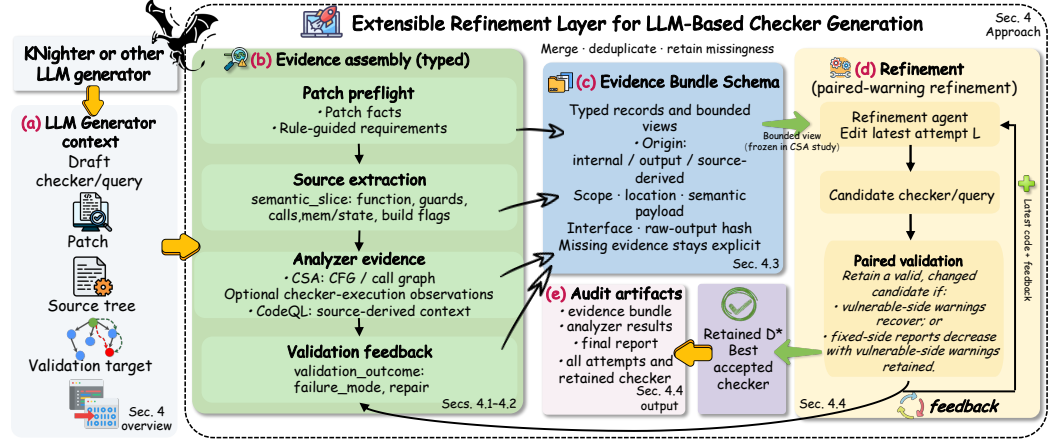


Fig. 2. **SEMWEAVER architecture.** Evidence assembly (green) feeds a provenance-labelled bundle (blue) to the editor (yellow). The retention gate (purple) keeps the best checker and returns feedback for the next iteration.

Algorithm 1 Paired-Warning Evidence-Guided Refinement

Require: detector D , patch P , source S , scope V , backends A , budget K

Ensure: refined detector D^* and audit report

```

1:  $R_0 \leftarrow \text{VALIDATE}(D, P, V)$ 
2:  $E_p, E_s \leftarrow \text{PREFLIGHT}(P), \text{EXTRACTSOURCE}(P, S, D)$ 
3:  $E_a \leftarrow \bigcup_{a \in A} \text{COLLECTNATIVE}(a, D, P, S)$    $E_o \leftarrow \text{FEEDBACK}(R_0)$ 
4:  $B \leftarrow \text{NORMALIZEMERGE}(E_p, E_s, E_a, E_o)$ 
5: if  $\text{PDS}(D, P, V)$  then return  $\text{AUDIT}(D, B)$ 
6:  $D^* \leftarrow D$    $L \leftarrow D$    $T \leftarrow \text{INITIALSTOPSTATE}$ 
7: for  $i = 1$  to  $K$  do
8:    $V_i \leftarrow \text{BOUNDEDVIEW}(B)$    $C_i \leftarrow \text{AGENTEDIT}(L, P, V_i)$ 
9:    $R_i \leftarrow \text{VALIDATEIFCOMPILABLE}(C_i, P, V)$ 
10:  if  $\text{ACCEPT}(C_i, D^*)$  then  $D^* \leftarrow C_i$ 
11:   $L \leftarrow C_i$    $E_i \leftarrow \text{FEEDBACK}(R_i)$ 
12:   $B \leftarrow \text{MERGE}(B, E_i)$ 
13:   $T \leftarrow \text{UPDATESTOPSTATE}(T, R_i, D^*)$ 
14:  if  $\text{STOP}(T, K)$  then break
15: end for
16: return  $\text{AUDIT}(D^*, B)$ 

```

► Equation 3

4.1 Evidence Selection and Exposure

The central design choice in evidence assembly (Figure 2b) is *what to collect*. Analyzer outputs can be large and heterogeneous. Passing them wholesale to the model dilutes the facts relevant to the checker under refinement. SEMWEAVER limits collection to patch-related source and presents selected evidence roles through bounded views.

Rule-guided preflight. PatchFacts Extractor parses affected functions, added guards, risky calls and type widenings. Evidence Planner uses a primary pattern from analysis metadata when available, or infers it from these facts. It assigns fixed priorities: patch anchoring is 100; lifetime and state requirements are 92 for double-free and 89 for use-after-free; null and bounds guards are 90. Requirements are sorted by decreasing priority, with evidence type breaking ties; the default

retains at most eight. Table 1 shows representative mappings. These priorities order collection requirements, rather than express confidence in individual facts. This component uses rules rather than a separately trained or prompted patch-mechanism classifier.

For the CSA comparison, collection is precomputed and the visible view is frozen before editing. The selected v43 implementation resolves compiler references into at most four function-local CFG witness cards, scoped by the patch and reports. It compares state associations and helper spelling with the checker, then appends available checker-execution observations. Additional evidence requests are disabled in this experiment. The general interface supports bounded, deduplicated role requests; their utility is outside this comparison.

Table 1. Representative evidence requirements by patch mechanism, using the persisted record types.

Patch mechanism	Possible record types	Backend
Cleanup edge or double free	allocation_lifecycle, state_transition, path_guard	CSA
Bounds widening or overflow	semantic_slice, dataflow_candidate, state_transition	CSA and CodeQL
Initialization or null dereference	state_transition, path_guard	CSA
Use-after-free or escape	allocation_lifecycle, call_chain	CSA and CodeQL
API misuse or missing check	semantic_slice, call_chain	CodeQL

4.2 Evidence Sources and Roles

Definition 1 (Analyzer-internal evidence). An observation is analyzer-internal when its payload comes from an analyzer-produced intermediate representation or from checker callbacks and state observed inside the analyzer, rather than from source text or a final warning alone. In SEMWEAVER, it also identifies the extraction interface and retained raw output, and is bound to the source revision and patch-related file and function.

Four collectors execute in sequence (Figure 2b, top to bottom).

Patch preflight identifies changed files, hunks and statements. This supplies the structural skeleton: *what changed* and *where*.

Source extraction resolves patch files and records bounded contexts around the changed code, including enclosing functions, branch guards, callee signatures, memory and state operations, and build flags. This provides local semantic context without dumping repository-scale source.

Analyzer evidence records what the backend produces. In the CSA study, Clang 18’s debug.DumpCFG and debug.DumpCallGraph emit UTF-8 dumps. The collector parses them into `semweaver.csa_cfg_snapshot.v1`, retaining the raw-output SHA-256 and source scope. They supply CFG branches, value bindings and call edges.

The implementation also observes instrumented CSA callbacks, abstract values and regions, and checker-state associations. Raw observations use JSONL schema `semweaver.checker_trace.v2`; bounded feedback uses `semweaver.trace_feedback.v1`. Original and instrumented scans must preserve diagnostics on both revisions. These are checker observations, not concrete executions of the vulnerable program.

We distinguish this material from two other origins. *Analyzer output* comprises diagnostics and alert counts. *Source-derived* records contain patch diffs, source windows or compile-database-assisted source extraction. A CodeQL record qualifies as internal evidence only when the database query, revision and returned tuples are retained; the CodeQL example in this study uses source-derived context.

Extraction must handle backend-specific output, instrumentation errors, capped observations and runs with no eligible callback. We check revision, architecture, source scope and diagnostic parity, and mark unavailable observations explicitly. Table 2 accounts for all 193 initial records across 39 subjects. The 81 internal records comprise 45 guard, 27 state-transition and 9 allocation-lifecycle records. The 61 source-derived records in this cohort are patch diffs; none is a source-window fallback. Per-attempt availability is reported in RQ3.

Table 2. Evidence primitives and provenance in the audited 39-case CSA cohort. One record may contain several CFG primitives; counts are by record origin, not primitive.

Origin	What the persisted record contains	Records
Analyzer-internal	Clang CFG guards, value bindings and call edges; raw-output and scope binding	81
Analyzer output	CSA diagnostics and paired validation reports; no internal state	51
Source-derived	Patch-diff records here; other interfaces may emit source-window fallbacks	61

Validation feedback records whether the detector compiles, executes, warns on the vulnerable revision and remains noisy on the fixed revision. It supplies the failure symptom for the next edit.

The seven persisted types are `patch_fact`, `semantic_slice`, `dataflow_candidate`, `call_chain`, `path_guard`, `allocation_lifecycle` and `state_transition`. They organize changed statements, bounded source context, possible propagation, call context, control conditions, resources and abstract state, respectively. When a selected role yields no evidence, the bundle retains an explicit missing-evidence marker rather than treating the absence silently, so the refinement agent knows what was requested but unavailable.

4.3 Evidence Bundle Schema

The normalizer merges all collected facts into the EvidenceBundle shown in Figure 2c. Each fact becomes an EvidenceRecord carrying a type, the contributing analyzer, scope, source location, payload, provenance, and an optional EvidenceSlice (the compact semantic content consumed by the agent). The schema separates program facts from analysis rules in the spirit of relational analyzer frameworks [2, 5, 24], but uses a smaller role vocabulary tailored to checker refinement. Formally:

$$\begin{aligned}
 B &= \text{Norm}(E_p \cup E_s \cup E_a \cup E_v), \\
 E_a &= E_{\text{CSA}} \cup E_{\text{CodeQL}}, \\
 r \in B.\text{records} &= \langle \text{type}, \text{analyzer}, \text{scope}, \\
 &\quad \text{location}, \text{payload}, \text{provenance}, \text{slice} \rangle.
 \end{aligned} \tag{1}$$

Here E_p contains patch facts, E_s bounded source slices, E_a analyzer representations or outputs, and E_v validation outcomes. The normalizer removes duplicates, records contributing analyzers and retains missing-evidence markers. Provenance identifies each record as `analyzer_internal`, `analyzer_output` or `source_derived`. The evidence type and origin are separate: the same role may hold a backend-produced record or a source-derived fallback. This decouples the bundle from any single backend representation.

4.4 Paired-Warning Refinement

The refinement agent receives the current detector, the patch, baseline validation notes and a bounded evidence view. The tool-mediated loop draws on LLM agent and self-refinement architectures, with actions restricted to evidence requests, detector edits and validation commands [40, 50, 53, 62]. The CSA comparison makes zero additional evidence requests.

Model and programmatic steps. The model proposes checker edits and may supply patch-analysis metadata or enabled role requests. Preflight rules, normalization, raw-output hashing, source binding, compilation, paired replay and acceptance are programmatic. The editor prompt gives the current and starting checkers, patch, source and evidence context, symptoms on both revisions, and backend-specific editing and validation rules. Repair prompts contain the actual compiler diagnostics. The artifact retains English templates and rendered exchanges, with missing historical exchanges marked.

Retention gate. A changed candidate must compile and execute normally on both revisions. Equation 3 then retains a recovered vulnerable-side warning, or fewer fixed-side reports while preserving vulnerable-side warning presence. Fixed-side silence with a vulnerable warning achieves the stronger PDS outcome. Native refinement additionally checks required hits on controlled array fixtures; Section 5 describes this guard and the warning-level metrics.

Feedback loop. When a candidate fails to compile or improve the retained result, its code and feedback remain available for the next attempt. The agent edits the latest attempt, while D^* stores the best retained checker. Algorithm 1 gives the loop; the protocol specifies its stopping conditions and treatment of interrupted executions. CSA checkers are compiled and replayed against both revisions, while CodeQL queries run over the corresponding databases.

Adding a backend. An adapter provides registration, evidence and provenance mapping, dependencies, execution and result decoding, and paired validation. Shared modules still contain backend branches. The four CodeQL-specific modules occupy 2,438 physical lines, including comments and generation code. With an analyzer and patch pair already available, we estimate that a familiar developer using Codex could complete a basic adapter within an afternoon; developer time was not measured. New native-evidence semantics and broad validation require additional work. A new bug mechanism needs a role mapping and fixtures for safe and unsafe cases, with a new extractor where a required relation is unavailable.

5 Evaluation

We evaluate whether analyzer evidence helps SEMWEAVER recover vulnerable-side warnings and reduce fixed-side reports. The evaluation focuses on patch-local and patch-related-file behavior rather than whole-kernel bug discovery.

5.1 Research Questions

RQ1: KNighter-derived checkers. How does SEMWEAVER compare with KNighter’s refinement loop on vulnerable-version warning coverage, fixed-side report burden and paired discrimination?

RQ2: Cross-backend feasibility and cost. Can the refinement layer operate on an archived CodeQL query associated with an earlier automatic generation run, and what integration and execution costs are observable?

RQ3: Evidence contribution. How do the native and no-internal variants differ, and what do their edits and reported gains establish?

RQ4: Model sensitivity. How does the workflow perform with three editor configurations on the same subset under common reply and output ceilings?

5.2 Experimental Setup

Subjects and benchmarks. We organize the evaluation into four experiments (E1–E4). E1 uses KNighter-derived Linux CSA checkers under patch-related object validation. From 286 artifact records on 60 commits, 241 records on 39 commits lack an upstream refinement-status row. We select one nonempty repaired-source artifact per commit by upstream TP, then TN, then the lowest checker index. This fixed, non-random rule selects the inputs; independent paired replay supplies their starting outcomes. All 39 enter KNighter refinement and both SEMWEAVER variants, including cases that remain noisy or fail to improve. Initially, 15 checkers warn on both revisions and 24 are silent on both. Their zero starting PDS is the study’s entry state.

The 15 initially noisy checkers also form a complete report-refinement stratum. Its results reuse the full-cohort outputs. For the 24 silent starts, KNighter’s no-report branch compiles without a model edit, whereas SEMWEAVER attempts recovery. We report these branches as the systems execute them.

The selected implementation is v43, chosen after comparing two configurations on six experimental subjects. Compatible executions on those subjects are reused in E1; the configuration-selection cases overlap the evaluation. The artifact retains those trials and their reuse checks. G04 uses its architecture-correct ARM64 scope and evidence after repair of an earlier wrong-object scan.

E2 uses an archived CodeQL input associated with an earlier automatic generation run, independently of KNighter, for ImageMagick (CVE-2017-12877, coders/mat.c). All three refinement decodes start from this same query. We report it separately from CSA. Earlier 20-row results from mixed and manual inputs are excluded. E3 compares native and no-internal on all 39 main pairs and repeats a 12-subject subset; E4 uses the same 12 subjects, native only, with three decodes per model (Figure 3).

Subset membership follows the initial scan status. We sort SHA-256 hashes of a fixed artifact-recorded salt plus case ID within the original 14 noisy and 25 silent cases, then take the first 4 and 8. A later scan repair corrects G08 from 0/0 to 2/2, leaving the same membership with 5 noisy and 7 silent subjects. The selection uses no refinement outcomes or vulnerability-class stratification; it contains no bounds cases.

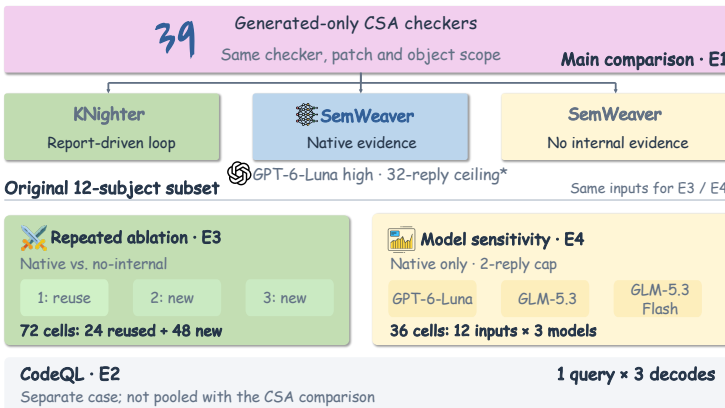


Fig. 3. **Experimental design and reuse.** E3’s first decode reuses E1. The E4 panel depicts its first decode (36 cells); two further decodes add 72 cells, for 108 records on the same 12 inputs. Cells are not independent bugs. *Audited natural-stop reuses retain original caps; shared ceilings do not imply equal realized effort.

Baselines and variants. We hold the generated detector fixed so that the comparison follows edits to the same checker rather than a fresh generation draw. E1 native, no-internal and KNighter use the same requested GPT-6-Luna-high editor and a 32-reply ceiling. KNighter retains its report-driven algorithm with five reports and two outer attempts, and its emitted checkers receive independent scans. The 24 no-report branches consume no reply. Two natural one-response No-FP stops are audited reuses that retain their original eight-reply ceiling; both terminate before either ceiling. We report actual replies and warning counts alongside the common maximum.

Within SEMWEAVER, no-internal removes native compiler and checker-execution views while keeping the checker, patch, frozen source, build and paired-scan feedback, and stopping rule. Native receives the bounded CFG witness and mismatch views and available dynamic observations. E3 repeats the original 12 subjects at 32 replies and 16,384 output tokens per call. E4 uses GPT-6-Luna high, GLM-5.3 high and GLM-5.3-Flash high, with three decodes per model, 2 replies per decode and 65,536 output tokens per call. Actual GLM effort fields are retained. Earlier default-effort, smaller-output and outside-subset histories are kept separate. E4 has no no-internal arm across models.

Experimental procedure. For each sample, we collect evidence, refine the detector and validate it on both revisions. Each permitted reply counts as an attempt. Compilable changed candidates receive independent paired replay and are retained under Equation 3. The next call receives the latest attempted code and feedback; the best retained detector is stored separately. The loop stops when a vulnerable warning and fixed-side silence are achieved, after three valid validations without improvement, after three consecutive invalid model attempts, or at the reply cap.

We preserve failed attempts and consumed prefixes when repairing execution problems. Thinking-only replies consume budget. For one GLM reply containing an explicit action and three edits but no explanatory summary, network-disabled replay supplied an empty summary and reproduced the model-specified code with no new call. Healthy zero-callback probes use static evidence. Compilation failures, format failures, unavailable observations, provider interruptions and invalid scans remain distinct; no manual checker completion enters the scored results.

Environment. CSA uses Clang/LLVM 18.1.3 and Python 3.12.3. CodeQL uses version 2.26.4, codeql/cpp-all 8.0.3 and no-build databases. Each Linux subject uses its patch commit and parent, allyesconfig, architecture-correct objects, and the same scope for starting, baseline and refined checkers. Manifests record code, evidence and raw paired outputs. The recorded model labels and high-effort requests identify the tested services. Both Responses adapters omit the configured temperature field, and no fixed seed is used.

Use of generative AI in the study. The editor models generated the checker and query edits evaluated in this study. Codex assisted with the development of experimental tools.

5.3 Metrics

We report vulnerable-version warning presence and fixed-side report burden. Let $n_v(D)$ and $n_f(D)$ count warnings in the same patch-related object scope on the vulnerable and fixed revisions. The stronger outcome, *patch-discriminating success* (PDS), requires a vulnerable-side warning and fixed-side silence:

$$\begin{aligned} H_v(D, P, V) &= \#[\text{alerts}(D, V_{\text{vuln}}, P) > 0], \\ S_f(D, P, V) &= \#[\text{alerts}(D, V_{\text{fix}}, P) = 0], \\ \text{PDS}(D, P, V) &= H_v(D, P, V) \wedge S_f(D, P, V). \end{aligned} \tag{2}$$

We distinguish two useful improvements. *Warning recovery* makes an initially silent vulnerable revision warning-positive, possibly with fixed-side noise. *Report reduction* lowers fixed-side reports

while preserving vulnerable-side warning presence. A detector that loses an existing vulnerable warning is rejected. A comparative disadvantage means that another arm retains a better result, which may still be an improvement over the starting checker.

For retained detector D^* and normally paired-executed candidate C , the improvement rule is:

$$\text{Accept}(C, D^*) = \text{AnalyzerOK}(C) \wedge H_v(C) \wedge (\neg H_v(D^*) \vee n_f(C) < n_f(D^*)) \wedge Q(C). \quad (3)$$

The editor requires a changed artifact before paired assessment. Q preserves required hits on controlled array fixtures in native and is identically true for no-internal. This native-only guard records zero rejections in E1/E3.

For precision, recall and $F_1 = 2TP / (2TP + FP + FN)$, each vulnerable revision is a positive and its fixed revision a negative, classified by warning presence. These metrics cover 78 patch-version decisions. Fixed-side report counts measure inspection burden. Neither measure adjudicates individual bug sites: G08, for example, warns on allocation failure rather than the registration-failure leak repaired by its patch. The qualitative follow-up in Section 6.5 examines target association separately; failed scans remain unscored.

6 Results

6.1 RQ1: KNightDerive Checkers

Table 3 reports the complete 39-subject comparison. All three refinement portfolios retain the 15 initially warning-positive vulnerable revisions. Among the 24 silent starts, native SEMWEAVER recovers eight, no-internal recovers six, and KNightDerive makes no recovery edit under its no-report policy. The anonymous artifact lists every subject and its paired counts.

Table 3. CSA comparison: (a) all 39 subjects; (b) the complete preexisting 15-case fixed-noisy stratum, reusing the same outputs. TP, FP and FN are version-warning decisions; fixed reports measure burden and replies count received responses. Color shades 1 – F_1 at .25, .50 and .75, not significance.

(a) Full cohort: 39 subjects

Condition	Version outcomes				F_1	Burden and effort	
	PDS	TP	FP	FN		Fixed reports	Replies
Unmodified	0	15	15	24	.435	73	0
KNightDerive loop	2	15	13	24	.448	69	179
SEMWEAVER, native	5	23	18	16	.5750	123	150
SEMWEAVER, no internal	8	21	13	18	.5753	68	133

(b) Noisy 15: all methods retain TP=15, PDS=2 and $F_1 = .6977$

	KNightDerive loop	Native	No internal
Fixed reports	69	44	68

However, the additional warning coverage comes with more fixed-side reports. Native produces 123, compared with 68 for no-internal and 69 for KNightDerive. Its precision is 0.561, versus 0.618 for no-internal; their F_1 values are nearly equal. G12 accounts for 73 native fixed-side reports at distinct locations. Its broader matching produces 74 vulnerable reports and 73 fixed reports, showing why warning recovery and report reduction need to be read together.

The initially noisy group gives a different view (Table 3b). On all 15 subjects, native reduces fixed reports by 25 (36.2%) against KNightDerive and 24 (35.3%) against no-internal. The reduction is concentrated in G24's 27 → 2 fixed reports; other small differences offset one another. All methods have the same binary fixed positivity and F_1 in this group. The follow-up in Section 6.5 finds

that G24’s surviving reports concern another lifecycle, so the count reduction does not establish preservation of its intended target.

6.2 RQ2: Cross-Backend Feasibility

Table 4 reports three automatic decodes of the archived ImageMagick CodeQL query. The starting query returns 0/0 rows on the vulnerable and fixed revisions. Two decodes recover 8/0; the third returns 22/22. The successful edit corrects relative-path matching while keeping the exact file, function and local-variable filters. A second query already starts at 2/0 and is excluded from the gain count. This experiment exercises query refinement separately from the KNighter CSA comparison.

Table 4. Three automatic CodeQL decodes of one query. Counts are output rows on the vulnerable and fixed revisions; time is recorded agent-round duration, excluding separate paired replay.

Decode	Rows (V,F)	Replies	Tokens	Agent time (s)
1	8/0	1	15,122	87.87
2	8/0	1	14,345	42.74
3	22/22	8	110,282	516.77

Second-backend costs. Three warm paired replays per query reproduce 0/0 before refinement and 8/0 after it. Median query-plus-decode time is 12.57 s initially and 36.51 s after refinement with two threads; peak process RSS is 976/1,078 MiB. Database-creation log intervals span 31/30 s, excluding unlogged startup. These are execution measurements in the existing environment. Developer labor and cold installation were not measured.

6.3 RQ3: Contribution of Analyzer Evidence

Table 2 accounts for the initial records. During refinement, 103 of 150 unique native-chain attempts have usable hash-bound checker observations. The remaining attempts are unavailable because of candidate compilation (11), unsupported context (18), zero scoped callbacks (13), a reproduced candidate crash (1), or capped optional observations (4). Zero observations are treated as absence of evidence.

Turning to the refinement outcomes, native has six comparative benefits, five disadvantages and 28 ties or other outcomes across the 39 subjects. The edits include cleanup-wrapper recognition in G19, nested dereference and binding recovery in G25, and handling of null values, regions and frames in G24. For example, adding the lowered cleanup name `__free_kfree` in G19 changes the counts on the vulnerable and fixed revisions to 15/0, while no-internal remains at 0/0. Conversely, no-internal recovers symbolic-region handling in G23 and persistent-field release recognition in G38. In G12, removing CPU-role restrictions broadens matching and produces the noise discussed above.

To examine variation across decodes, Figure 4 shows all three paired decodes on the original 12 subjects. Five subjects tie in every decode; seven have mixed directions. None benefits comparatively in all three. G19’s native 15/0 repeats, but no-internal catches up in the third decode. G24’s main report reduction does not repeat. G14 improves its own start from 3/3 to 2/2 but trails no-internal’s 2/1; later decodes tie. The disadvantages in G05/G23/G38 arise because native stays at its initial 0/0 while no-internal recovers warnings. These outcomes show variation in what the editor can recover, even when the accepted artifact preserves its own initial warning signal.

Subject	Decode 1 (E1)			Decode 2			Decode 3		
G04	=	0/0	: 0/0	=	0/0	: 0/0	=	0/0	: 0/0
G08	=	1/1	: 1/1	—	2/2	: 1/1	=	1/1	: 1/1
G09	+	1/1	: 0/0	=	1/1	: 1/1	=	1/1	: 1/1
G13	=	0/0	: 0/0	=	0/0	: 0/0	=	0/0	: 0/0
G14	—	2/2	: 2/1	=	2/1	: 2/1	=	2/1	: 2/1
G15	=	9/9	: 9/9	=	9/9	: 9/9	=	9/9	: 9/9
G19	+	15/0	: 0/0	+	15/0	: 0/0	=	15/0	: 15/0
G24	+	2/2	: 27/27	=	3/3	: 3/3	=	27/27	: 27/27
G25	+	2/2	: 0/0	=	0/0	: 0/0	=	0/0	: 0/0
G36	=	1/1	: 1/1	=	1/1	: 1/1	=	1/1	: 1/1
G37	=	0/0	: 0/0	=	0/0	: 0/0	=	0/0	: 0/0
G38	—	0/0	: 1/0	=	1/0	: 1/0	=	1/0	: 1/0

Native V/F : no-internal V/F. + benefit; — disadvantage; = tie/other.

Recovery: native 4 / control 1; report reduction: native 1 / control 2; ties/other 28.

Fig. 4. E3: all 36 paired decodes on 12 subjects; cells compare native and no-internal counts on the vulnerable and fixed revisions. Decode 1 reuses E1; the remaining decodes add 48 condition cells. Color and sign encode the comparative direction.

6.4 RQ4: Sensitivity Across Models

Table 5 uses the same 12 subjects with three native-editor configurations and three decodes each. All configurations retain the five initial vulnerable warning signals and recover additional versions in the first decode. The other decodes show how much those results vary.

Table 5. Limited-budget native sensitivity: three decodes on the same 12 subjects per model, not full-39 robustness. F_1 is scored per decode; mean and sample SD summarize the three values. Fixed reports count warning occurrences on fixed revisions.

Editor	F_1 1	F_1 2	F_1 3	Mean	SD	Fixed reports 1/2/3
GPT-6-Luna high	.560	.615	.500	.558	.058	40/36/41
GLM-5.3 high	.583	.615	.667	.622	.042	16/13/14
GLM-5.3-Flash high	.692	.560	.667	.640	.070	11/41/40

Received replies across the three decodes are 24/23/24 for GPT-6-Luna, 23/23/21 for GLM-5.3 and 21/23/21 for GLM-5.3-Flash. Their PDS counts are 1/2/0, 2/2/4 and 4/1/4, respectively; identical warning counts on the vulnerable and fixed revisions across all three runs occur on 8, 3 and 4 of the 12 subjects. Capped, format-invalid and compile-failed attempts remain charged; a candidate that cannot complete a valid paired scan is not scored as an improvement. GLM-5.3-Flash has the highest mean F_1 . However, its first-decode fixed-report advantage does not recur; GLM-5.3 has fewer fixed-side reports in all three decodes. The same 12 subjects, two-reply ceiling and native-only arm limit what can be inferred about other checkers or the contribution of evidence across models.

Controlled source replay. Beyond model variation, we examine source changes in three selected original subjects, G01, G08 and G09; their unchanged-source controls reproduce the original paired counts. Local renaming and inert no-ops leave counts unchanged in these cases. Under the equivalent guard rewrite, native retains vulnerable-side warning presence in eight of nine case-change pairs: G01 falls from 2/2 to 0/0, G08 rises from 1/1 to 2/2, and G09 remains 1/1. Thus the

loss of two fixed-side reports in G01 is also a loss of its vulnerable warning, while G08’s preserved warning costs another fixed-side report. These controlled changes identify source-form sensitivity without treating the three subjects as independent natural evolution events.

6.5 Qualitative Findings

To validate whether warning-level improvements reflect genuine semantic understanding, we conduct three supplementary analyses. First, a target and report audit inspects all 11 non-tied native/no-internal comparisons across 362 report occurrences and 22 code diffs, classifying each report by target association and source support. Second, mechanism probes construct six controlled variants per case for G12, G19, G24 and G25 (48 files, 96 scans), and a separate intervention replaces native-observation lines with missing markers to test whether evidence removal changes the model’s editing decisions. Third, a controlled source replay rescans three subjects under local renames, inert no-ops and equivalent guard rewrites to measure source-form sensitivity without re-invoking the model.

To explain the native and no-internal differences, we inspect all 11 non-tied comparisons and their reported locations. The quantitative results match several distinct changes in the checkers. Among the six native benefits, G12/G19/G25 have bounded source-supported target reports, G09 has conditional support, and G24/G35 retain no supported or conditional target report. Among the five no-internal advantages, G01/G23/G38 have source support and G05/G14 conditional support. This source audit explains the warning counts without changing the primary scores.

Both G01 variants repair recognition of the governing loop, but only no-internal checks the preceding capacity guard. G19 adds a lowered cleanup name. G23 changes symbol and nested-member lookup; G25 changes sink recognition and destination binding. These repairs make the intended operation visible to the checker. Other edits relax predicates: G05 preserves initialization uncertainty, G09 broadens cleanup handling, G12 drops actor restrictions, and G38 combines field-region preservation with a looser usage gate. G35’s new trigger instead reports on an initialized overflow parameter outside the patched target.

The surviving report population matters as much as the count. G14 chooses different event predicates and retains reports about different lifetime properties. G24 changes region, NULL and frame tests, removing 25 reports per side; its two survivors concern owner destruction outside the target surviving-owner lifecycle. Some removed target reports are conditional. Thus this case shows lower inspection burden, but neither proves a valid target witness was lost nor establishes that the target was preserved.

Frozen perturbations. On the six positive and six negative fixtures per mechanism, retained native G12 reports at a target location in all positives and no negatives. This is within its annotation model; the diagnostic’s remote-write note points to a local reset. G19 is silent on its adapted-wrapper suite. Its separate canonical control produces one supported early-exit report and one wrong-scope allocator-return report, and it misses the renamed wrapper. G25 reports at target locations in all six positives and five negatives. The starting and no-internal artifacts are silent for these three cases. For G24, the starting and no-internal artifacts report at target locations in all six positives and six negatives, with 108/108 total reports. Native has no frozen target-line hits and 18/12 total reports. Secondary inspection supports some guard warnings outside that line list; the original location scores remain unchanged.

Selected-clue intervention. All three full-evidence G25 decisions peel the outer casts around the sink expression; no withheld decision does. Two full decisions also change bookkeeping. After the separate compiler-only completion, full recovers target-line presence with negative-side noise,

while withheld remains silent (Table 6). The full-evidence reports still need inspection. All 34 reports from repetitions 2/3 follow $\text{argc} \leq 1$, which selects a concrete non-NULL object. Repetition 1 has six outer count-access reports that admit a source-feasible NULL completion, without an explicit NULL-return event, alongside unsupported reports.

Table 6. GLM-5.3-Flash bounded-adaptor terminal artifacts after separate compiler-only completion. Each cell gives full and withheld results. Primary suites have six positive and six negative fixtures; G19 lowering has two of each and benign has four negatives. Hits and alerts are final-location presence, not validated diagnoses. Original strict-schema and adaptor failures remain separate (Section 6.5).

Draw	G19			G25		
	Primary positive hits	Lowering positive hits	Benign alerts	Positive hits	Negative alerts	Negative reports
1	0/6	1/2	0/2	6/0	5/0	28/0
2	0/6	1/2	0/2	6/0	5/0	10/0
3	6/0	2/0	1/0	6/0	6/0	6/0

G19 exposes a different tradeoff. Full repetitions 1/2 retain name-family tests, while withheld generalizes registration; repetition 3 reverses primary coverage. The broader matchers can warn on benign cleanup. Across these artifacts, supported owning-scope exits coexist with reports at callee returns and a write-only cleanup mistaken for a read.

The negative cases are equally important. Correcting the callback or expression recognized by a checker can activate a useful check, yet leave the object, guard or owning scope wrong. Even the final source line can contain both a safe pointer-field read and the optional-object dereference of interest. Conversely, a source-supported guard warning can fall outside the chosen sink-line list. Examining the edit and its diagnostic together explains why recovery, target-line agreement and semantic support sometimes disagree.

7 Discussion

Scope of improvement. SEMWEAVER refines a detector with respect to its motivating patch. A human-written checker could use the same interface with a compatible backend, patch and validation pair. We study generated inputs, for which the patch supplies a common refinement target. A checker may know *where* to warn yet miss *why* the warning should disappear after the fix. Our results show that refinement also has an earlier task: making the checker recognize the relevant operation in the analyzer’s representation. G19 needs the lowered cleanup callback; G25 needs to look through an expression wrapper. These are useful repairs, but the remaining scope and path conditions still determine whether a warning is justified.

Evidence as context compression. The relevant condition may span source code and analyzer outputs. Passing the full repository to the model is impractical, while passing only the patch may leave out the representation seen by the checker. SEMWEAVER collects a provenance-tagged summary of patch-relevant facts and makes it available with validation feedback. The interface follows the general idea of refining an artifact under additional program evidence [12, 39, 41, 42, 55]. Its usefulness varies by case: the no-internal arm also repairs representation problems in G01, G23 and G38, and nearly matches native F_1 overall.

Backend interfaces. CSA and CodeQL offer different checker formats and evidence interfaces. The automatic CodeQL case shows that the same editing and paired-validation workflow can operate on a query. It retains source-derived context and inherited patch-local filters. The recorded

warm execution costs and existing adapter footprint describe that implementation; broader transfer and measured developer effort remain open.

Failure modes. Over-pruning is one failure mode, but retaining a warning is not enough either. In G24, fewer fixed-side reports coexist with surviving reports outside the intended lifecycle. In G19, a broader cleanup matcher can report at a callee return while the caller’s variable is still live. In G25, a recovered target-line warning can follow an explicitly non-NULL path. These cases suggest reporting the actual checker edit, the property supported by its diagnostic, and the change in inspection burden together. This separates successful recognition from the semantic conditions still needing repair.

Generalization. The motivating example survives the tested renamings, capacities and equivalent guard forms. Those tests informed its refinement, and the additional four-mechanism suites are controlled source reductions. The replay of three original subjects adds a different check: local renames and no-ops preserve their counts, but an equivalent guard rewrite silences G01 on both versions and increases G08’s fixed-side reports. Transfer to unseen projects and vulnerability instances requires a separate evaluation.

8 Threats to Validity

Internal. An error in evidence collection, LLM editing or validation could produce a spurious improvement. We retain raw outputs, code hashes, failed attempts and unavailable observations; all scored checker edits are automatic. Configuration selection used six experimental subjects, with compatible executions reused in E1. Native context also bundles several components and includes a fixture guard absent from no-internal, although that guard rejects no E1/E3 candidate. The checkpoint intervention varies selected clues while retaining other native context. Its post-response adapter and compiler-only continuations are separate from the strict first decisions, and equal reply ceilings do not ensure equal realized compute. Model aliases, omitted temperature fields and the absence of a fixed seed leave decoding variance uncontrolled.

Construct. The primary metrics describe warnings on vulnerable and fixed revisions under the same patch-related scope, not target-level precision and recall or confirmed false-positive bugs. G08 concerns a different leak path, and G24’s retained warnings concern another lifecycle. Final-line matching cannot distinguish two objects or a feasible and infeasible path on the same line; the follow-up adds AI-assisted interpretation of source and paths without changing primary scores. These judgments lack independent human calibration; a source-feasible completion is not a run-time witness. G19 and G25 use different mixed-observation conventions, so their labels are not pooled as accuracy. Some historical zero-scan records lack complete stdout.

External. The 39 subjects are selected representatives from one generator’s Linux artifacts. The noisy and silent starting groups exercise different refinement behavior, including KNighter’s no-report policy. The 11 non-ties, four perturbation mechanisms and two checkpoints form nested, selected scopes; G12 models access annotations and G24 logical retirement rather than executed races or physical kernel UAF. The motivating fixtures informed its edits. The model study uses three decodes per configuration on the same 12 subjects, with no bounds cases; CodeQL remains one patch-local example. The controlled source replay covers three selected original subjects and does not represent independent natural evolution.

Conclusion. Repeated decodes and transformed fixtures are correlated observations, not additional independent bugs. Native and no-internal F_1 are nearly tied, and no repeated subject benefits

comparatively in all three decodes. Our findings explain local repairs and failures; they do not estimate their prevalence or establish a rule predicting when native evidence will help.

9 Related Work

Static Analysis and Checker Languages. Static-analysis systems provide the substrate that SEMWEAVER refines against. Path-sensitive analyzers, abstract interpreters, and symbolic engines expose guards, states, paths, and diagnostics reusable as evidence [3, 6, 13, 15] and deploy at industrial scale [4, 7, 49]. Declarative languages offer a complementary approach by querying program facts through relational rules and points-to specifications [2, 5, 24], while semantic patching encodes recurring transformations over systems code [43]. SEMWEAVER introduces no new analyzer or query language. It treats generated checker code as the artifact to be refined.

Vulnerability Detection and Patch Evidence. Static vulnerability detection has long used taint and data-flow reasoning [1, 25, 36], graph and AST representations [58, 59], and learning-based token-, graph-, slice-, and line-level models [17, 34, 35, 47], though datasets and evaluations show that semantic robustness remains difficult [8, 16]. Clone- and patch-driven systems use historical fixes as compact witnesses [22, 28, 33]. These motivate our use of patches to scope the refinement task.

LLM-Based Checker Synthesis. Code models and repository agents support program generation and analysis [11, 19]; LLMDFA decomposes data-flow reasoning and synthesizes tools [57]. Security studies show why generated code requires validation [26, 44, 45]. KNighter [60] is the closest workflow: it generates, compiles, edits and revalidates checkers using patch and report context. QLCoder [56] synthesizes CodeQL queries from CVE metadata via an agentic loop with execution feedback and structured tool access. SEMWEAVER adds bounded compiler and checker observations to that setting and attempts warning recovery on silent starts.

Feedback-Guided Repair. The loop also relates to repair and synthesis under compiler, test, verifier or learned feedback [12, 27, 30, 38, 39, 41, 42, 55], and to LLM work on iterative feedback, tool use and search [40, 50, 53, 61, 62]. Weak validation proxies and repair overfitting are established concerns [46, 54]. Our follow-up examines their consequences for edited static checkers, where a warning can return at the expected line while its object, scope or path remains wrong.

Analyzer Context and Alert Triage. LLift uses analyzer reports and post-constraints [31]; Chapman et al. interleave analyzer intermediates and prompting for error specifications [9]; IRIS infers taint specifications with CodeQL [32]. LLM4PFA reasons about report feasibility [14], and ZeroFalse enriches reports before adjudication [21]. VulGenie combines patch dependencies, attack and defense flows, and API-rule synthesis [10]. These studies already use analyzer context. SEMWEAVER instead studies how such context changes an existing checker’s implementation and the reports it produces, retaining both the useful edits and their remaining semantic errors.

10 Conclusion

We presented SEMWEAVER, a post-synthesis refinement layer that combines analyzer evidence, patch context and paired validation to refine LLM-generated vulnerability checkers. On 39 KNighter-derived Linux checkers, it recovers more vulnerable-version warning signals than KNighter’s report-driven loop, while nearly matching no-internal F_1 and producing more fixed-side reports overall. The complete non-tie audit and selected follow-up experiments explain both useful local repairs and the conditions they leave unresolved. Repeated model decodes and controlled source changes also show where those repairs vary or fail to transfer across source

forms. Analyzer evidence can help a checker recognize the relevant operation; deciding whether that operation is unsafe still requires the right object, guard, path and scope.

Acknowledgments

In paper writing, the authors used generative AI tools (ChatGPT and Claude) solely for language polishing and clarity-oriented editing.

Data Availability

Our code and data are available at <https://anonymous.4open.science/r/SemWeaver-FD1E/>.

References

- [1] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. FlowDroid: Precise Context, Flow, Field, Object-Sensitive and Lifecycle-Aware Taint Analysis for Android Apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 259–269. doi:10.1145/2594291.2594299
- [2] Pavel Avgustinov, Oege de Moor, Michael Peyton Jones, and Max Schäfer. 2016. QL: Object-Oriented Queries on Relational Data. In *Proceedings of the 30th European Conference on Object-Oriented Programming (ECOOP)*. 2:1–2:25. doi:10.4230/LIPIcs.ECOOP.2016.2
- [3] Thomas Ball and Sriram K. Rajamani. 2002. The SLAM Project: Debugging System Software via Static Analysis. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 1–3. doi:10.1145/503272.503274
- [4] Al Bessey, Ken Block, Ben Chelf, Andy Chou, Bryan Fulton, Seth Hallem, Charles Henri-Gros, Asya Kamsky, Scott McPeak, and Dawson Engler. 2010. A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World. *Commun. ACM* 53, 2 (2010), 66–75. doi:10.1145/1646353.1646374
- [5] Martin Bravenboer and Yannis Smaragdakis. 2009. Strictly Declarative Specification of Sophisticated Points-to Analyses. In *Proceedings of the 24th ACM SIGPLAN Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*. 243–262. doi:10.1145/1640089.1640108
- [6] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 209–224. https://www.usenix.org/legacy/event/osdi08/tech/full_papers/cadar/cadar.pdf
- [7] Cristiano Calcagno, Dino Distefano, Jérémy Dubreil, Dominik Gabi, Pieter Hooimeijer, Martino Luca, Peter O’Hearn, Irene Papakonstantinou, Jim Purbrick, and Dulma Rodriguez. 2015. Moving Fast with Software Verification. In *Proceedings of the 7th NASA Formal Methods Symposium (NFM)*. 3–11. doi:10.1007/978-3-319-17524-9_1
- [8] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2022. Deep Learning Based Vulnerability Detection: Are We There Yet? *IEEE Transactions on Software Engineering* 48, 9 (2022), 3280–3296. doi:10.1109/TSE.2021.3087402
- [9] Patrick J. Chapman, Cindy Rubio-González, and Aditya V. Thakur. 2024. Interleaving Static Analysis and LLM Prompting. In *Proceedings of the 13th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis*. 9–17. doi:10.1145/3652588.3663317
- [10] Bofei Chen, Shuang Liao, Lei Zhang, Chibin Zhang, Mathias Payer, and Yuan Zhang. 2026. Patch-Guided Vulnerability Detection: Extracting Java API Security Rules via Attack–Defense Cross-Analysis. In *35th USENIX Security Symposium (USENIX Security 26)*. USENIX Association, Baltimore, MD, 1847–1866. <https://www.usenix.org/conference/usenixsecurity26/presentation/chen-bofei>
- [11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). doi:10.48550/arXiv.2107.03374
- [12] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. 2000. Counterexample-Guided Abstraction Refinement. In *Proceedings of the 12th International Conference on Computer Aided Verification (CAV)*. 154–169. doi:10.1007/10722167_15
- [13] Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, David Monniaux, and Xavier Rival. 2005. The ASTRÉE Analyzer. In *Proceedings of the 14th European Symposium on Programming (ESOP)*. 21–30. doi:10.1007/978-3-540-31987-0_3

- [14] Xueying Du, Kai Yu, Chong Wang, Yi Zou, Wentai Deng, Zuoyu Ou, Xin Peng, Lingming Zhang, and Yiling Lou. 2025. Minimizing False Positives in Static Bug Detection via LLM-Enhanced Path Feasibility Analysis. *arXiv:2506.10322* <https://arxiv.org/abs/2506.10322>
- [15] Dawson Engler, David Yu Chen, Seth Hallem, Andy Chou, and Benjamin Chelf. 2001. Bugs as Deviant Behavior: A General Approach to Inferring Errors in Systems Code. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*. 57–72. doi:10.1145/502034.502041
- [16] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*. 508–512. doi:10.1145/3379597.3387501
- [17] Michael Fu and Chakkrit Tantithamthavorn. 2022. LineVul: A Transformer-Based Line-Level Vulnerability Prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*. 608–620. doi:10.1145/3524842.3528452
- [18] GitHub. 2026. CodeQL Documentation. <https://codeql.github.com/docs/>. Accessed: Jun. 27, 2026.
- [19] Jinyao Guo, Chengpeng Wang, Xiangzhe Xu, Zian Su, and Xiangyu Zhang. 2025. RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 21083–21100. <https://proceedings.mlr.press/v267/guo25n.html>
- [20] Sarah Heckman and Laurie Williams. 2011. A Systematic Literature Review of Actionable Alert Identification Techniques for Automated Static Code Analysis. *Information and Software Technology* 53, 4 (2011), 363–387. doi:10.1016/j.infsof.2010.12.007
- [21] Mohsen Iranmanesh, Sina Moradi Sabet, Sina Marefat, Ali Javidi Ghasr, Allison Wilson, Iman Sharafaldin, and Mohammad A. Tayebi. 2025. ZeroFalse: Improving Precision in Static Analysis with LLMs. *arXiv:2510.02534* <https://arxiv.org/abs/2510.02534>
- [22] Jiyong Jang, Abeer Agrawal, and David Brumley. 2012. ReDeBug: Finding Unpatched Code Clones in Entire OS Distributions. In *Proceedings of the 2012 IEEE Symposium on Security and Privacy*. 48–62. doi:10.1109/SP.2012.13
- [23] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. 2013. Why Don’t Software Developers Use Static Analysis Tools to Find Bugs?. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, San Francisco, CA, 672–681. doi:10.1109/ICSE.2013.6606613
- [24] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. 2016. Soufflé: On Synthesis of Program Analyzers. In *Proceedings of the 28th International Conference on Computer Aided Verification (CAV)*. 422–430. doi:10.1007/978-3-319-41540-6_23
- [25] Nenad Jovanovic, Christopher Kruegel, and Engin Kirda. 2006. Pixy: A Static Analysis Tool for Detecting Web Application Vulnerabilities. In *Proceedings of the 2006 IEEE Symposium on Security and Privacy*. 258–263. doi:10.1109/SP.2006.29
- [26] Raphaël Khoury, Anderson R. Avila, Jacob Brunelle, and Baba Mamadou Camara. 2023. How Secure Is Code Generated by ChatGPT?. In *Proceedings of the 2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 2445–2451. doi:10.1109/SMC53992.2023.10394237
- [27] Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. 2013. Automatic Patch Generation Learned from Human-Written Patches. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 802–811. doi:10.1109/ICSE.2013.6606626
- [28] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. 2017. VUDDY: A Scalable Approach for Vulnerable Code Clone Discovery. In *Proceedings of the 2017 IEEE Symposium on Security and Privacy*. 595–614. doi:10.1109/SP.2017.62
- [29] Ted Kremenek and Dawson Engler. 2003. Z-Ranking: Using Statistical Analysis to Counter the Impact of Static Analysis Approximations. In *Proceedings of the 10th International Static Analysis Symposium (SAS)*. 295–315. doi:10.1007/3-540-44898-5_16
- [30] Claire Le Goues, Michael Dewey-Vogt, Stephanie Forrest, and Westley Weimer. 2012. A Systematic Study of Automated Program Repair: Fixing 55 out of 105 Bugs for \$8 Each. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*. 3–13. doi:10.1109/ICSE.2012.6227211
- [31] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. 2024. Enhancing Static Analysis for Practical Bug Detection: An LLM-Integrated Approach. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1, Article 111 (2024), 26 pages. doi:10.1145/3649828
- [32] Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/582d4e27fa24168f3af1f4582655034b-Abstract-Conference.html
- [33] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Hanchao Qi, and Jie Hu. 2016. VulPecker: An Automated Vulnerability Detection System Based on Code Similarity Analysis. In *Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC)*. 201–213. doi:10.1145/2991079.2991102

- [34] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2022. SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* 19, 4 (2022), 2244–2258. doi:10.1109/TDSC.2021.3051525
- [35] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. In *Proceedings of the 25th Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2018.23158
- [36] V. Benjamin Livshits and Monica S. Lam. 2005. Finding Security Vulnerabilities in Java Applications with Static Analysis. In *Proceedings of the 14th USENIX Security Symposium*. 271–286. https://www.usenix.org/legacy/events/sec05/tech/full_papers/livshits/livshits.pdf
- [37] LLVM Project. 2026. Clang Static Analyzer Documentation. <https://clang-analyzer.llvm.org/>. Accessed: Jun. 27, 2026.
- [38] Fan Long and Martin Rinard. 2015. Staged Program Repair with Condition Synthesis. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 166–178. doi:10.1145/2786805.2786811
- [39] Fan Long and Martin Rinard. 2016. Automatic Patch Generation by Learning Correct Code. In *Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 298–312. doi:10.1145/2837614.2837617
- [40] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 46534–46594. doi:10.52202/075280-2019
- [41] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. 2016. Angelix: Scalable Multiline Program Patch Synthesis via Symbolic Analysis. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. 691–701. doi:10.1145/2884781.2884807
- [42] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. 2013. SemFix: Program Repair via Semantic Analysis. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 772–781. doi:10.1109/ICSE.2013.6606623
- [43] Yoann Padioleau, Julia L. Lawall, René Rydhof Hansen, and Gilles Muller. 2008. Documenting and Automating Colateral Evolutions in Linux Device Drivers. In *Proceedings of the 3rd ACM SIGOPS/EuroSys European Conference on Computer Systems*. 247–260. doi:10.1145/1352592.1352618
- [44] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy*. 754–768. doi:10.1109/SP46214.2022.9833571
- [45] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2785–2799. doi:10.1145/3576915.3623157
- [46] Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. 2015. An Analysis of Patch Plausibility and Correctness for Generate-and-Validate Patch Generation Systems. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA)*. Association for Computing Machinery, 24–36. doi:10.1145/2771783.2771791
- [47] Rebecca Russell, Louis Kim, Lei Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul Ellingwood, and Marc McConley. 2018. Automated Vulnerability Detection in Source Code Using Deep Representation Learning. In *Proceedings of the 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*. 757–762. doi:10.1109/ICMLA.2018.00120
- [48] Joseph R. Ruthruff, John Penix, J. David Morgenthaler, Sebastian Elbaum, and Gregg Rothermel. 2008. Predicting Accurate and Actionable Static Analysis Warnings: An Experimental Approach. In *Proceedings of the 30th International Conference on Software Engineering (ICSE)*. 341–350. doi:10.1145/1368088.1368135
- [49] Caitlin Sadowski, Jeffrey van Gogh, Ciera Jaspan, Emma Soderberg, and Collin Winter. 2015. Tricorder: Building a Program Analysis Ecosystem. In *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE)*. 598–608. doi:10.1109/ICSE.2015.76
- [50] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 68539–68551. doi:10.52202/075280-2997
- [51] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. 2012. AddressSanitizer: A Fast Address Sanity Checker. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*. USENIX Association, Boston, MA, 309–318. <https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany>
- [52] Srinivasan Shanmugam. 2024. drm/amd/display: Fix buffer overflow in get_host_router_total_dp_tunnel_bw(). Linux kernel commit 97cba232549b9fe7e491fb60a69cf93075015f29. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=97cba232549b9fe7e491fb60a69cf93075015f29> Commit metadata and source checked locally, September 27, 2026.

- [53] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 8634–8652. doi:10.52202/075280-0377
- [54] Edward K. Smith, Earl Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the Cure Worse than the Disease? Overfitting in Automated Program Repair. In *Proceedings of the 10th Joint Meeting of the European Software Engineering Conference and ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*. Association for Computing Machinery, 532–543. doi:10.1145/2786805.2786825
- [55] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 404–415. doi:10.1145/1168857.1168907
- [56] Claire Wang, Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. QLCoder: A Query Synthesizer For Static Analysis of Security Vulnerabilities. *arXiv preprint arXiv:2511.08462* (2025). doi:10.48550/arXiv.2511.08462
- [57] Chengpeng Wang, Wuqi Zhang, Zian Su, Xiangzhe Xu, Xiaoheng Xie, and Xiangyu Zhang. 2024. LLMDDFA: Analyzing Dataflow in Code with Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 131545–131574. doi:10.52202/079017-4181
- [58] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and Discovering Vulnerabilities with Code Property Graphs. In *Proceedings of the 2014 IEEE Symposium on Security and Privacy*. 590–604. doi:10.1109/SP.2014.44
- [59] Fabian Yamaguchi, Markus Lottmann, and Konrad Rieck. 2012. Generalized Vulnerability Extrapolation Using Abstract Syntax Trees. In *Proceedings of the 28th Annual Computer Security Applications Conference (ACSAC)*. 359–368. doi:10.1145/2420950.2421003
- [60] Chenyuan Yang, Zijie Zhao, Zichen Xie, Haoyu Li, and Lingming Zhang. 2025. KNighter: Transforming Static Analysis with LLM-Synthesized Checkers. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*. 655–669. doi:10.1145/3731569.3764827
- [61] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 11809–11822. doi:10.52202/075280-0517
- [62] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=WE_vluYUL-X